

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-210БВ-25

Студент: Старыгина А.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.10.26

Постановка задачи

Вариант 22.

Родительский процесс создаёт два дочерних процесса. Пользователь вводит имена двух файлов, затем произвольные строки. Родитель с вероятностью 80% отправляет строку в `pipe1`, иначе - в `pipe2`. Дочерние процессы инвертируют строки и пишут результат в свои файлы.

Общий метод и алгоритм решения

Для решения задачи использовались следующие системные вызовы:

- `fork()` - создание дочерних процессов;
- `pipe()` - организация неименованных каналов;
- `dup2()` - перенаправление стандартных потоков ввода-вывода;
- `open()` / `close()` - работа с файлами;
- `read()` / `write()` - обмен данными;
- `execlp()` - замена образа процесса;
- `waitpid()` - ожидание завершения потомков.

Алгоритм работы: родитель читает имена двух файлов, создаёт два канала и два дочерних процесса. В каждом ребёнке через `dup2` связывает `stdin` с соответствующим каналом, а `stdout` - с файлом, после чего запускает программу `child` через `execlp`. Родитель в цикле читает строки от пользователя и с вероятностью 80% пишет их в первый канал, иначе - во второй, используя `write`. Дочерний процесс посимвольно читает из `stdin`, накапливает строку до `\n`, инвертирует её и записывает в `stdout`. По завершении ввода родитель закрывает каналы на запись, что приводит к EOF у детей, и ожидает их через `waitpid`.

В рамках лабораторной работы мы разработали две отдельные программы на C++ - `parent` и `child`, взаимодействующие через неименованные каналы. Родительский процесс принимал от пользователя имена двух файлов, создавал каналы `pipe1` и `pipe2`, а затем с помощью `fork` и `exec` запускал два экземпляра дочерней программы, перенаправляя их стандартный ввод на чтение из каналов, а стандартный вывод - в соответствующие файлы. Пользователь вводил произвольные строки, которые родитель случайным образом (80% / 20%) распределял между каналами. Каждый дочерний процесс читал строки из своего канала, инвертировал их (переворачивал порядок символов) и записывал результат в свой файл. Все системные вызовы проверялись на ошибки, а по завершении ввода родитель корректно закрывал каналы и дожидался завершения потомков через `waitpid`.

Код программы

parent.cpp

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#include <sys/wait.h>
```

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <cstdlib>
```

```
#include <ctime>
```

```
#include <string>
```

```
#include <cstring>
```

```
void error(const char *msg)
```

```
{
```

```
    write(2, msg, strlen(msg));
```

```
    write(2, "\n", 1);
```

```
    exit(1);
```

```
}
```

```
std::string read_line(int fd)
```

```
{
```

```
    std::string line;
```

```
    char c;
```

```
while (true)

{

    ssize_t n = read(fd, &c, 1);

    if (n < 0)

        error("read error");

    if (n == 0)

        break;

    line.push_back(c);

    if (c == '\n')

        break;

}

return line;

}
```

```
int main()

{

    std::string file1 = read_line(0);

    if (!file1.empty() && file1.back() == '\n')

        file1.pop_back();

    std::string file2 = read_line(0);

    if (!file2.empty() && file2.back() == '\n')

        file2.pop_back();

}
```

```
int pipe1[2], pipe2[2];
```

```
if (pipe(pipe1) < 0)
```

```
    error("pipe1");
```

```
if (pipe(pipe2) < 0)
```

```
    error("pipe2");
```

```
int fd1 = open(file1.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
if (fd1 < 0)
```

```
    error("open file1");
```

```
int fd2 = open(file2.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
if (fd2 < 0)
```

```
    error("open file2");
```

```
pid_t pid1 = fork();
```

```
if (pid1 < 0)
```

```
    error("fork1");
```

```
if (pid1 == 0)
```

```
{
```

```
    if (dup2(pipe1[0], 0) < 0)
```

```
        error("dup2 pipe1 read");
```

```
    if (dup2(fd1, 1) < 0)
```

```
        error("dup2 fd1");
```

```
close(pipe1[1]);

close(pipe1[0]);

close(pipe2[0]);

close(pipe2[1]);

close(fd1);

close(fd2);

execlp("./child", "./child", nullptr);

error("exec child1");

}
```

```
pid_t pid2 = fork();

if (pid2 < 0)

    error("fork2");

if (pid2 == 0)

{

    if (dup2(pipe2[0], 0) < 0)

        error("dup2 pipe2 read");

    if (dup2(fd2, 1) < 0)

        error("dup2 fd2");

    close(pipe1[1]);

    close(pipe1[0]);

    close(pipe2[0]);

    close(pipe2[1]);

    close(fd1);

}
```

```
close(fd2);

execlp("./child", "./child", nullptr);

error("exec child2");
}
```

```
close(pipe1[0]);

close(pipe2[0]);

close(fd1);

close(fd2);
```

```
srand(time(NULL));
```

```
while (true)
{
    std::string line = read_line(0);

    if (line.empty())
        break;
```

```
int r = rand() % 100;

int fd = (r < 80) ? pipe1[1] : pipe2[1];
```

```
    ssize_t written = write(fd, line.c_str(), line.size());

    if (written < 0)

        error("write to pipe");

}
```

```
close(pipe1[1]);
```

```
close(pipe2[1]);
```

```
waitpid(pid1, nullptr, 0);
```

```
waitpid(pid2, nullptr, 0);
```

```
return 0;
```

```
}
```

child.cpp

```
#include <unistd.h>
```

```
#include <cstring>
```

```
#include <string>
```

```
#include <cstdlib>
```

```
void error(const char *msg)
```

```
{
```

```
    write(2, msg, strlen(msg));
```

```
    write(2, "\n", 1);

    exit(1);
}
```

```
int main()

{

    std::string line;

    char c;


    while (true)

    {

        ssize_t n = read(0, &c, 1);

        if (n < 0)

            error("read error");

        if (n == 0)

            break;


        if (c == '\n')

        {

            std::string rev(line.rbegin(), line.rend());

            rev.push_back('\n');

            ssize_t written = write(1, rev.c_str(), rev.size());

            if (written < 0)
```

```

        error("write error");

        line.clear();

    }

    else

    {

        line.push_back(c);

    }

}


if (!line.empty())

{

    std::string rev(line.rbegin(), line.rend());

    rev.push_back('\n');

    write(1, rev.c_str(), rev.size());

}


return 0;

}

```

Протокол работы программы

Тест 1:

\$./parent

out1.txt

out2.txt

hello

world

abc

\$ cat < out1.txt

olleh

cba

\$ cat < out2.txt

dlrow

Тест 2:

\$./parent

a.txt

b.txt

test

\$ cat < a.txt

tset

\$ cat < b.txt

Тест 3:

\$./parent

/nonexistent_dir/out.txt

b.txt

hello

open file1: No such file or directory

\$ echo \$?

1

Вывод

В ходе работы освоены системные вызовы `fork`, `pipe`, `dup2`, `exec`, `waitpid`. Программа корректно распределяет строки между двумя дочерними процессами, которые инвертируют их и пишут в файлы. Все ошибки обрабатываются, тесты пройдены. Основная сложность с которой я столкнулась, это - правильный порядок закрытия дескрипторов каналов после `fork`, иначе дочерние процессы зависали без EOF. Также пришлось вручную реализовывать построчное чтение из каналов из-за запрета `stdio.h`.